Sir Syed University
of Engineering & Technology

Operating System Lab

Project Report

Supervised by

Sir Waqas Akber

Operating Systems Lab Project

BS Computer Science, Batch 2022F

Group Members:

Name	Roll Number
Saad Ali	2022F-BCS-162
Hamza Khan	2022F-BCS-180
Muhammad Faisal	2022F-BCS-152
Muhammad Khubaib	2022F-BCS-191

_____

Supervisor

Project Report

❖ Scheduling Algorithm

1. Introduction

The First Come First Serve (FCFS) and Round Robin (RR) scheduling algorithms are two fundamental CPU scheduling techniques widely studied and implemented in operating systems. This project aims to provide a comprehensive understanding and practical implementation of both algorithms using a C# Windows Form GUI and SQL Server for data management. The application offers an intuitive and user-friendly interface, allowing users to interact seamlessly with the scheduling algorithms. By incorporating both free and premium features, the application caters to a wide range of users, enhancing their learning experience and providing valuable insights into CPU scheduling.

2. Project Objective

The primary objective of this project is to create a user-friendly application that demonstrates the functionality of both the FCFS and RR scheduling algorithms. The application should allow users to:

- Register and log in with unique credentials.
- Input and manage process data.
- Calculate and display key scheduling metrics such as completion time, turnaround time, and waiting time.
- Provide additional premium features such as Gantt Chart visualization and detailed breakdowns of each calculation step for paid users.
- Collect user feedback for further improvements.

3. Features

The project includes the following features:

a) User Authentication

- **Login**: Users log in by entering their username and password.
- **Registration**: New users can create an account by providing their name, a unique username, and a password.

b) Home Page

- **Poster Display**: A visual representation or poster of the FCFS and RR algorithms is displayed on the home page.

c) Process Management

- **Add Process**: Users can add process details (arrival time, burst time, process ID) which are displayed in a DataGridView.
- **Choose Algorithm:** Users can select either FCFS or RR scheduling algorithms.
- **Input Time Quantum:** Users can input the time quantum for the RR scheduling algorithm.
- **Calculate Metrics**: Calculates and displays completion time, turnaround time, and waiting time for each process, along with average turnaround time and average waiting time.
- **Clear Processes**: Allows users to clear the entered process data from the DataGridView.

d) Premium Features

- **Upgrade to Pro**: Users can upgrade to the pro version by paying $9 to access additional features.
- **Pro Features**: Includes Gantt Chart visualization and detailed breakdown of calculation steps.
- **Payment Integration**: Users can enter card details (Union, Visa, VisaMaster, Mastercard) to complete the upgrade.

e) Feedback

- **User Feedback**: Users can provide feedback about the application.

4. Problem Statements

- **User Authentication**: Ensuring secure and unique user registration and login.
- **Data Management**: Efficiently storing and retrieving user and process data using SQL Server.
- **Calculation Accuracy**: Accurately calculating and displaying scheduling metrics for both FCFS and RR algorithms.
- **User Interface**: Designing an intuitive and responsive GUI for a seamless user experience.
- **Payment Processing**: Securely handling payment transactions for upgrading users to the pro version.
- **Feature Access Control**: Differentiating between free and pro users to restrict or grant access to premium features

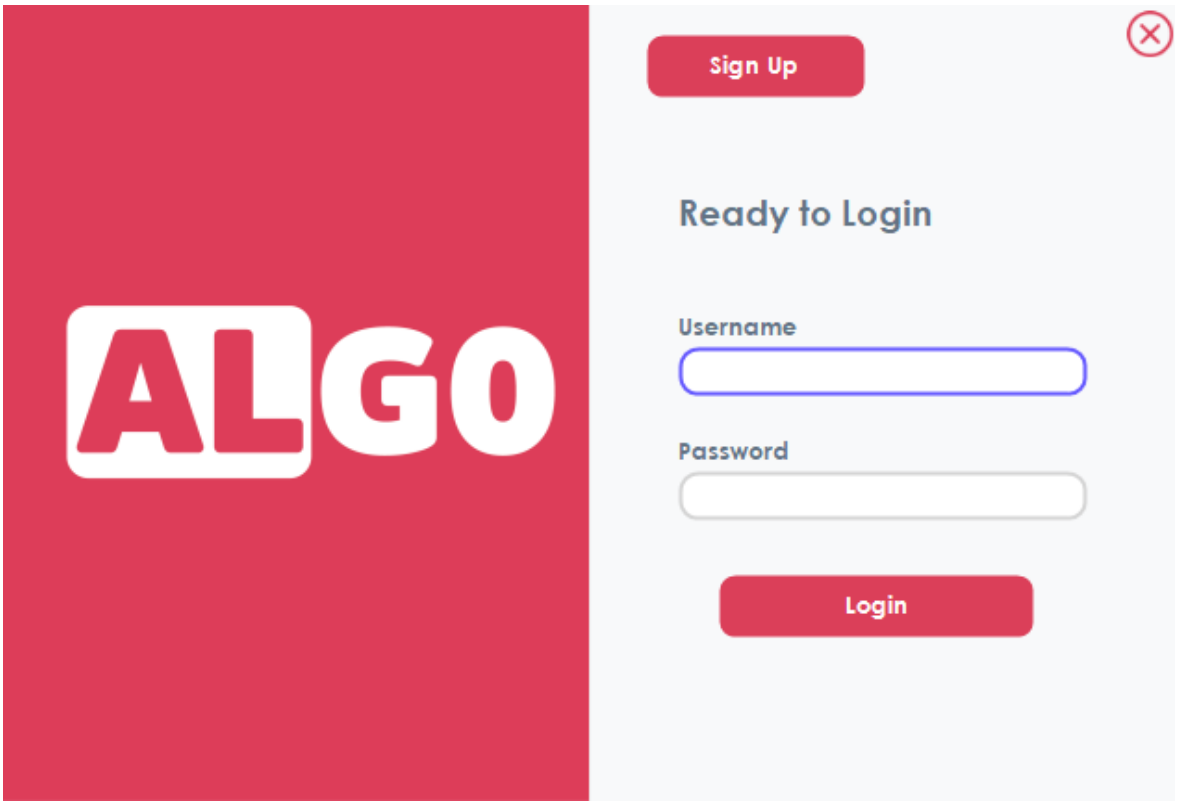
5. Technologies Used

- **Programming Language**: C#
- **Framework**: .NET Framework for Windows Forms
- **Database**: SQL Server for storing user credentials and process data
- **GUI Design**: Windows Forms for creating the user interface

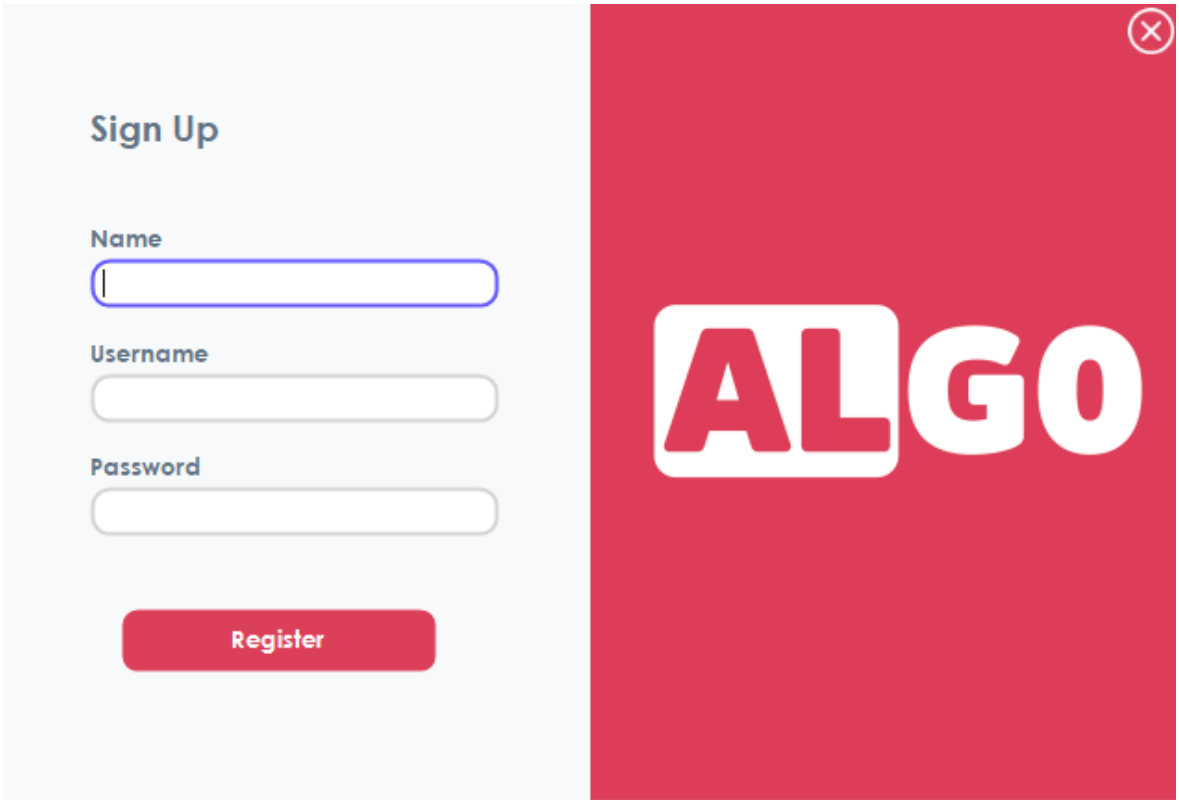
6. Conclusion

The Scheduling Algorithm project successfully demonstrates the implementation of both the FCFS and RR algorithms using a C# Windows Form application with SQL Server integration. The project achieves its objectives by providing essential scheduling functionalities along with a user-friendly interface. The additional premium features offer enhanced visualization and detailed breakdowns, making the application both educational and practical for users. Future enhancements could include adding more scheduling algorithms and further improving the user interface based on feedback.

Upon launching the application, the user is presented with the login form.

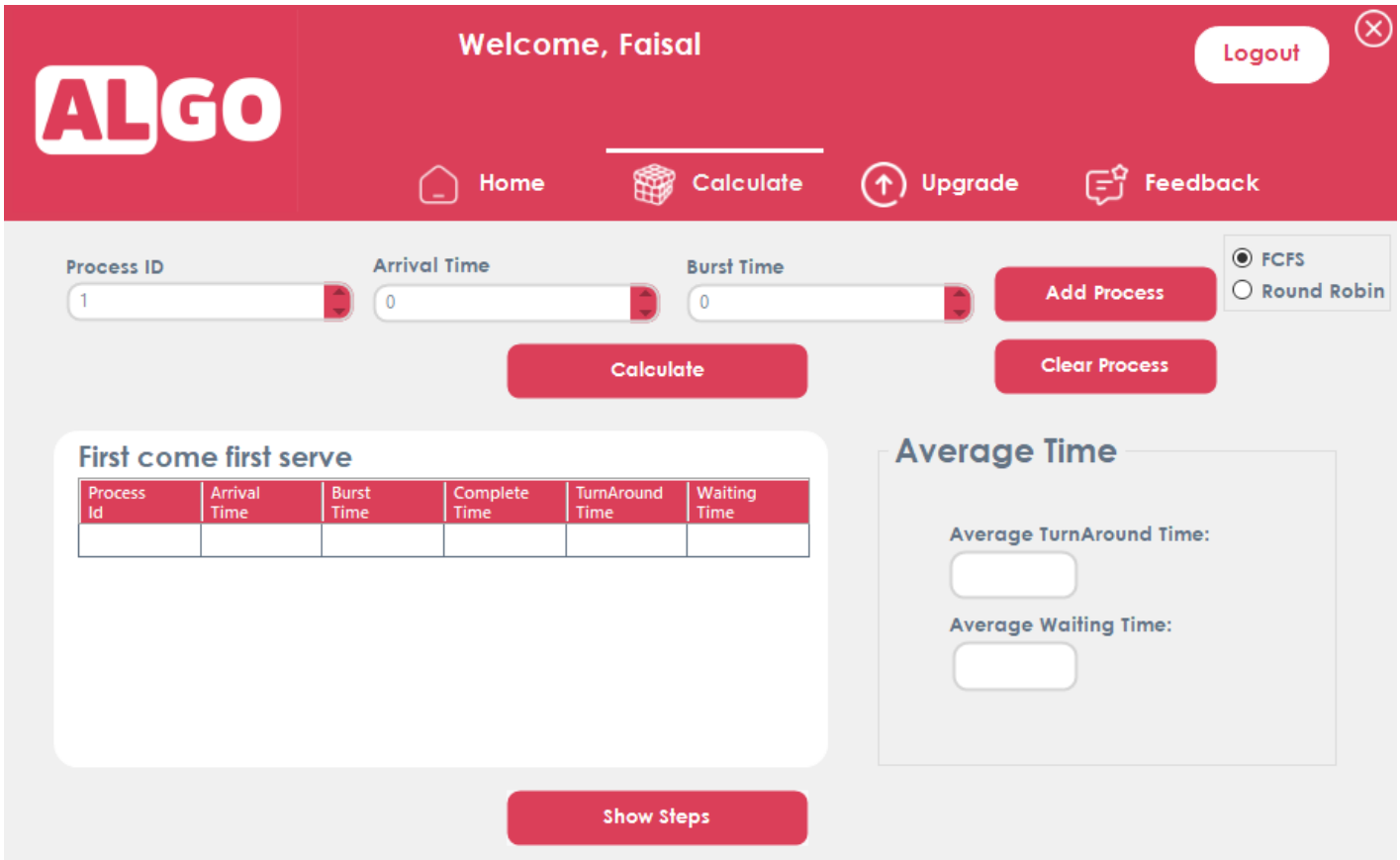


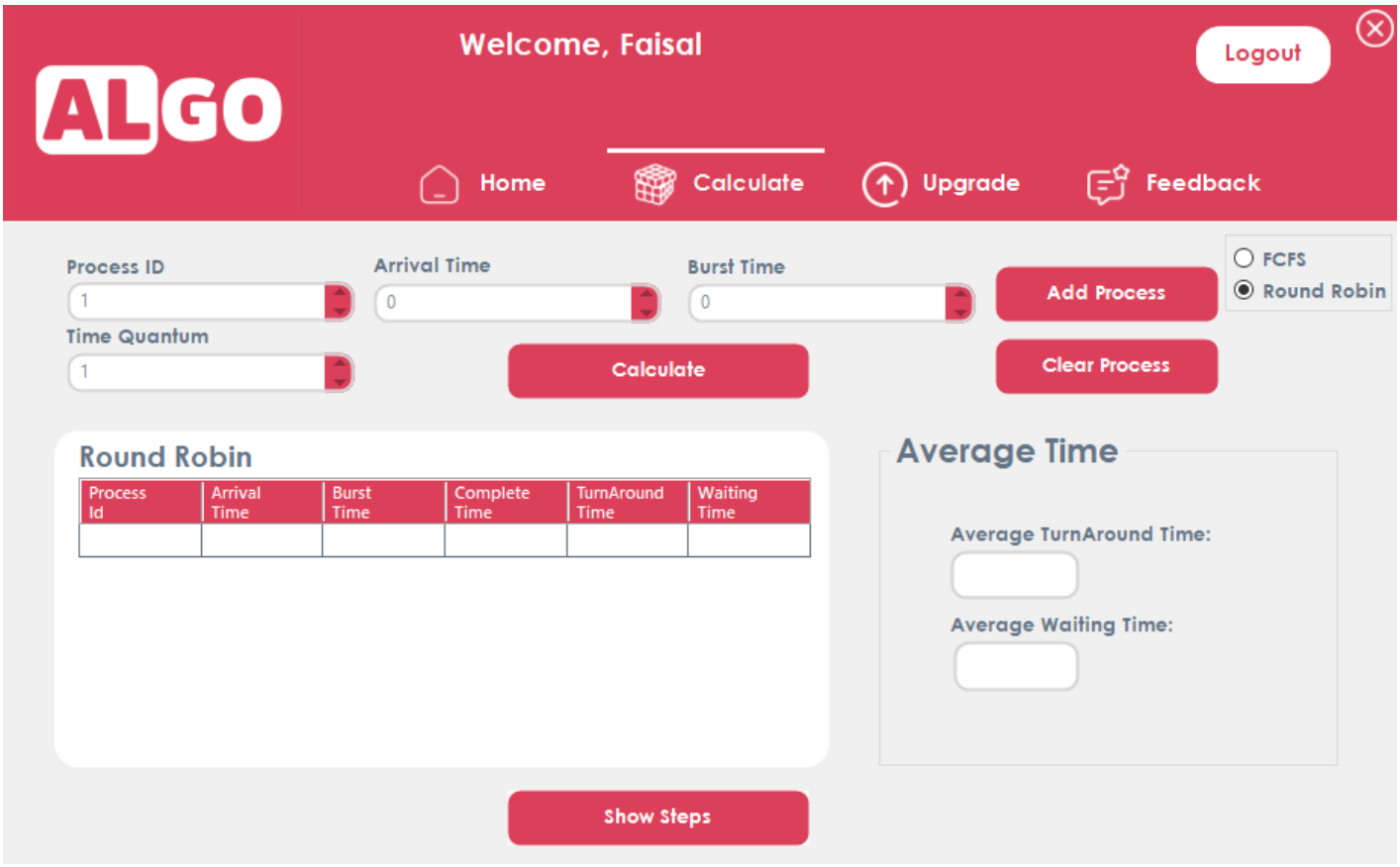
If the user has not already signed up, they can do so before logging in. The username must be unique. If the username is already taken, the application will display an error message.



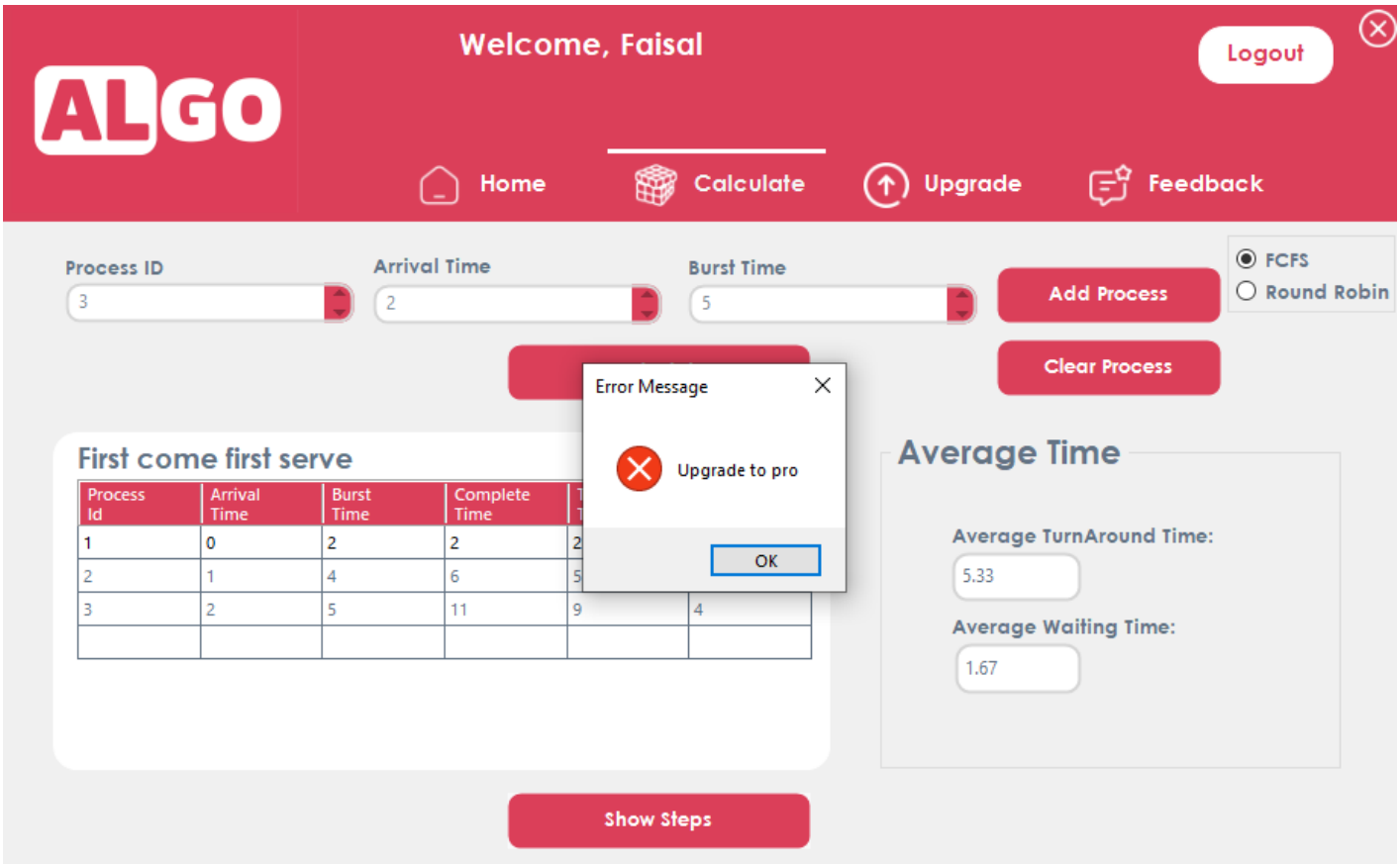
Once the user signs in, the home page is displayed, showcasing a poster for FCFS and Round Robin scheduling algorithms

When the user clicks the "Calculate" button, they can choose between FCFS (default) and Round Robin scheduling algorithms. They can add or clear processes. The application calculates and displays the waiting time, completion time, and turnaround time in a DataGridView, along with the average TAT and WT

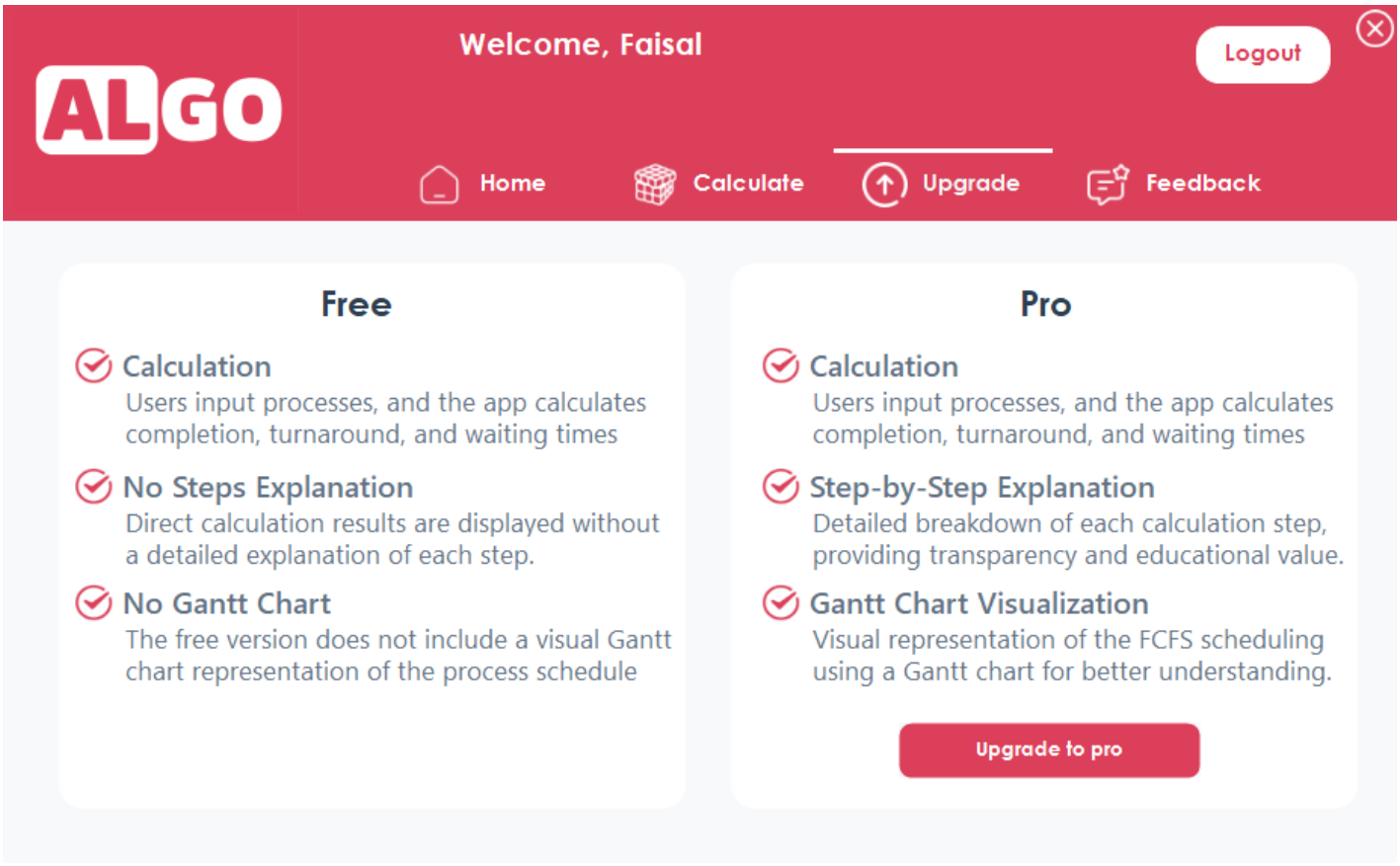




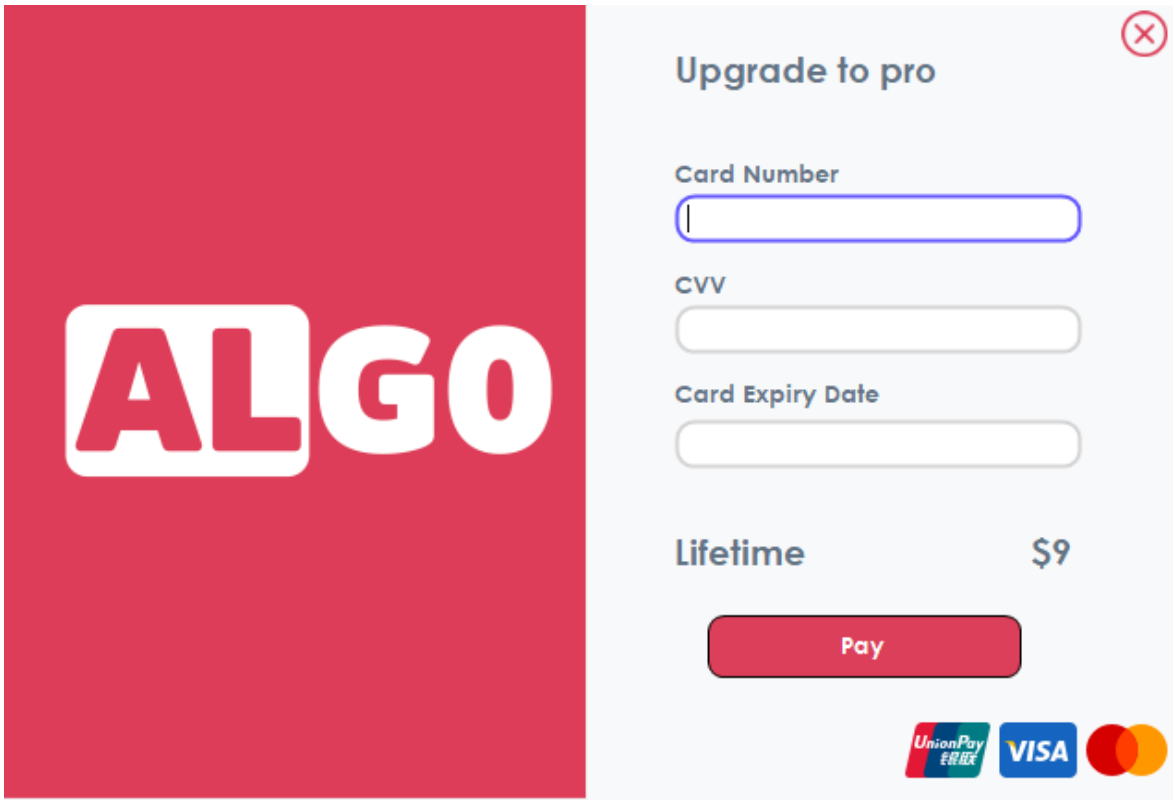
When the user clicks "Show Steps," if they have not upgraded to Pro, the application displays an error message instructing them to upgrade to Pro first



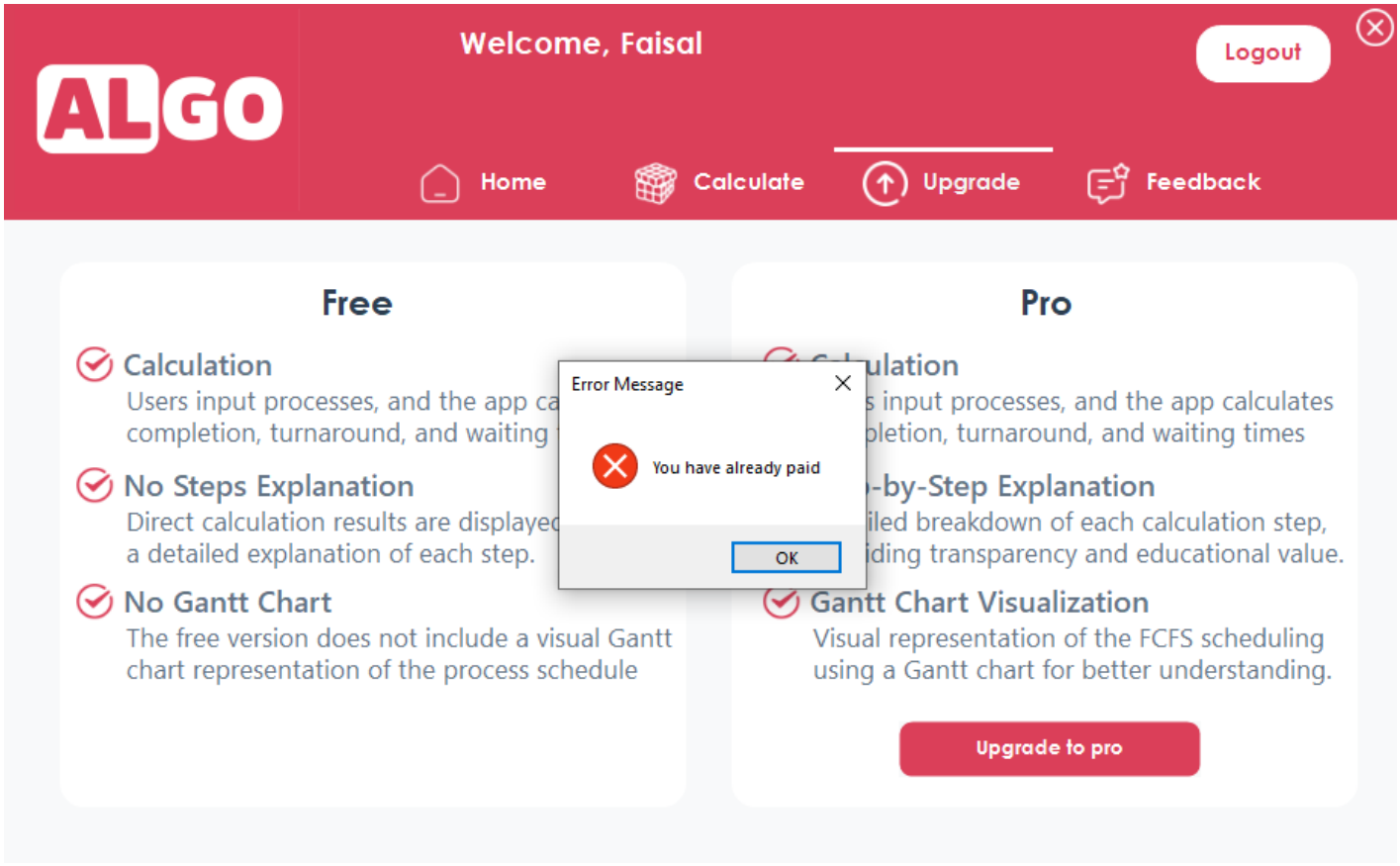
When the user clicks the upgrade button, a panel appears showing the benefits of the Pro version. The user can then upgrade by clicking the Pro button.



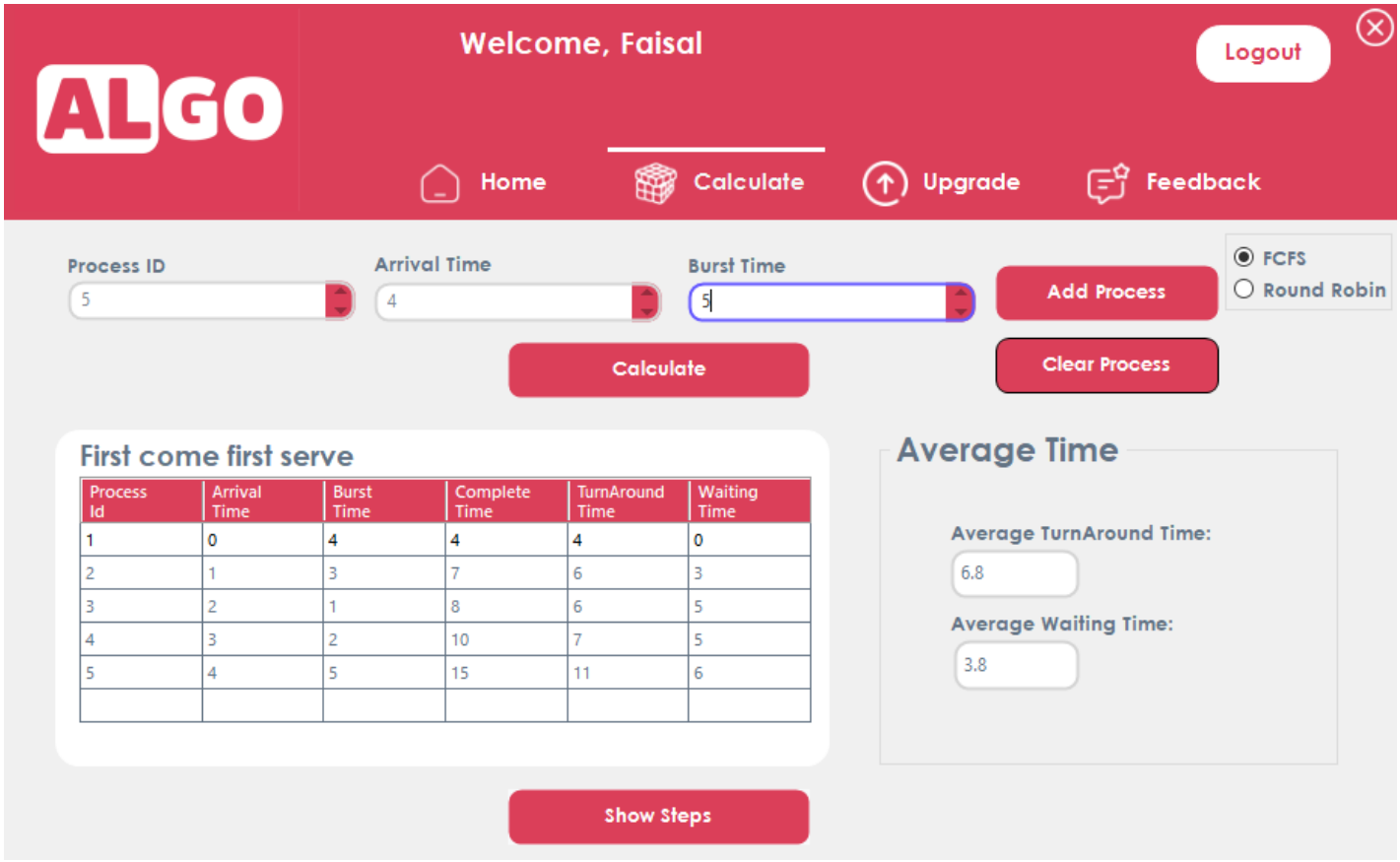
When the user clicks the "Upgrade to Pro" button, a new form appears where they can input their card number, CVV, card expiry date. The upgrade costs $9 for a lifetime subscription. Only UnionPay, Visa, Visa Master, and MasterCard are accepted.

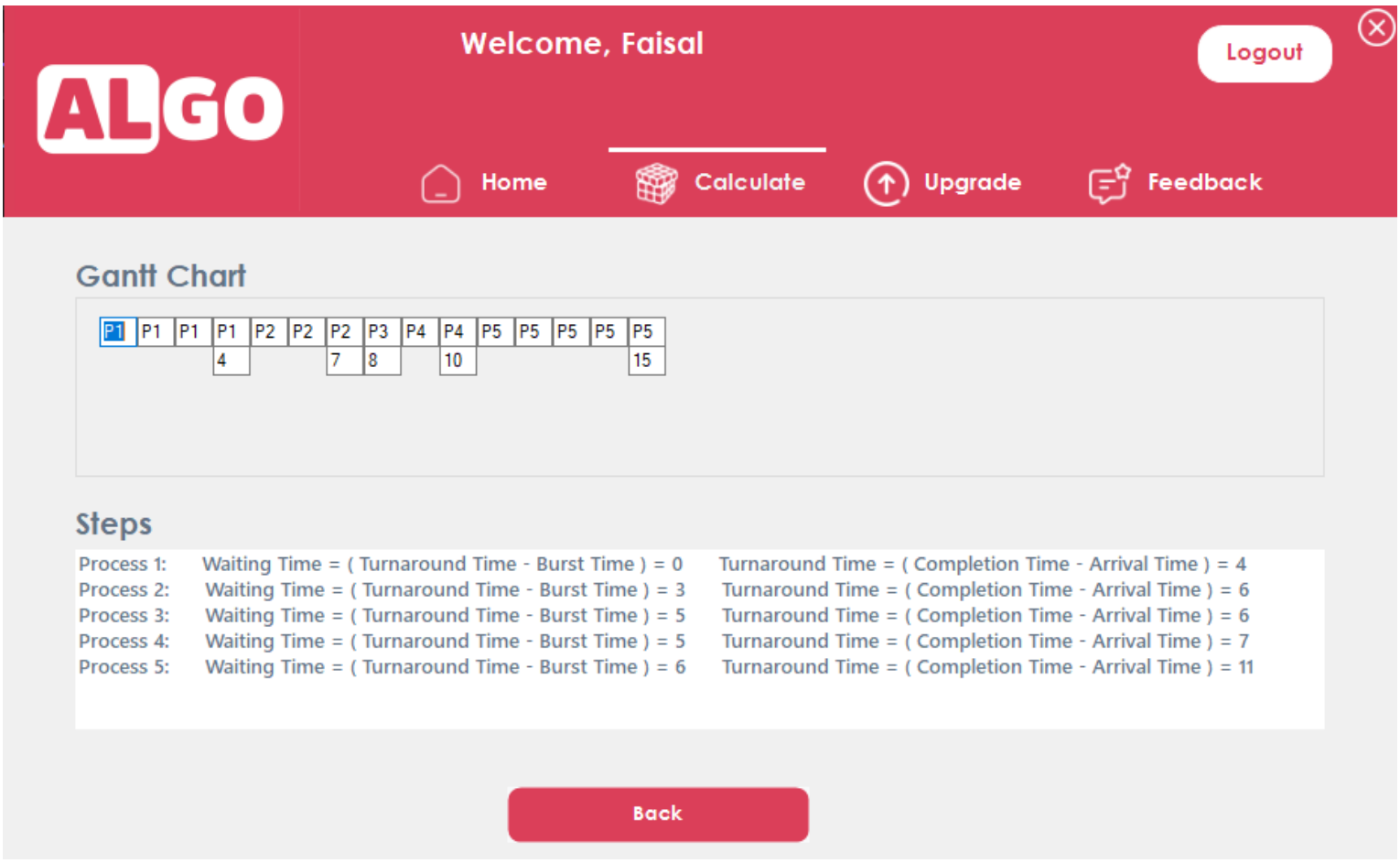


If the user has already paid and clicks the "Upgrade to Pro" button again, an error message will be displayed indicating that they have already paid.

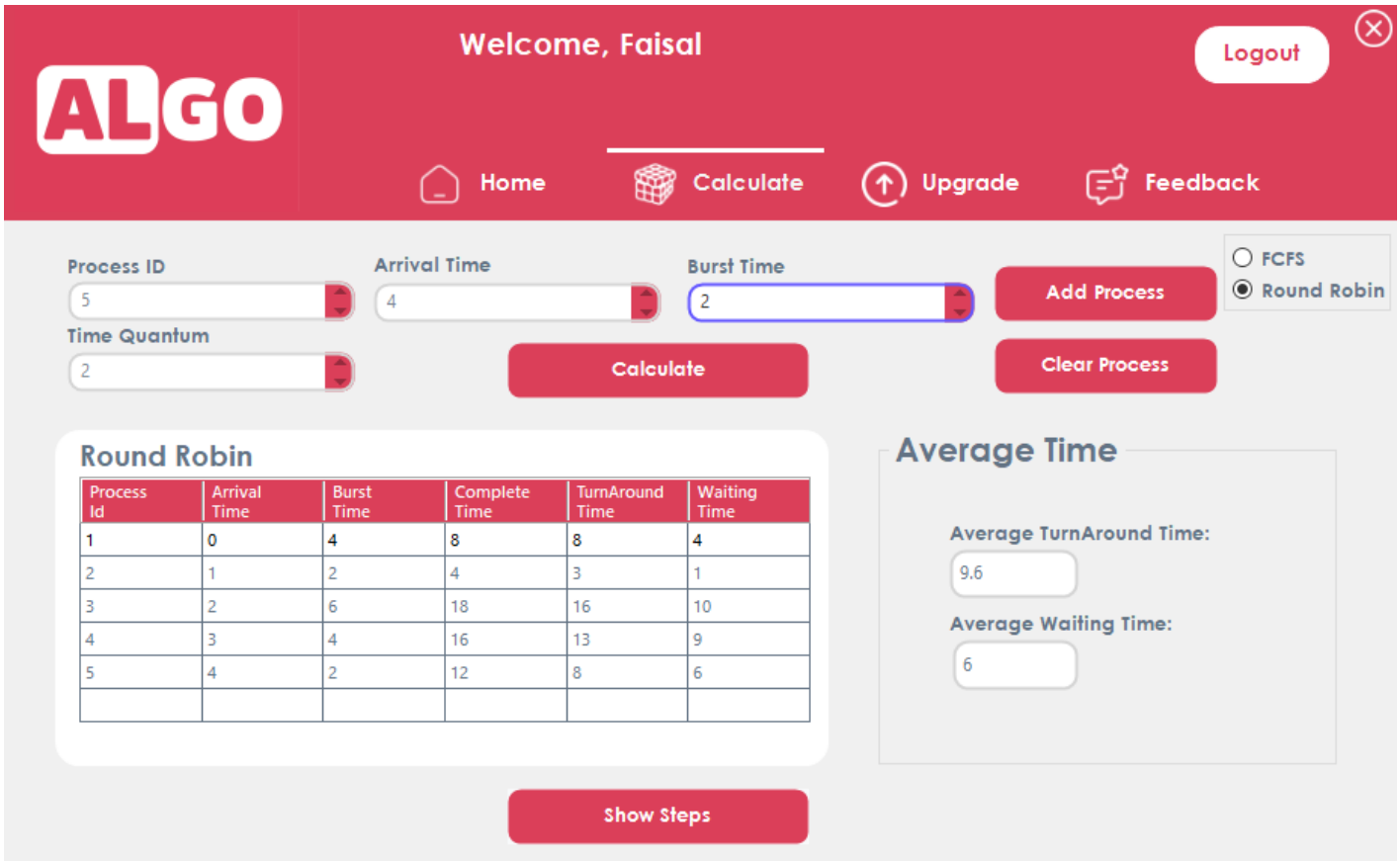


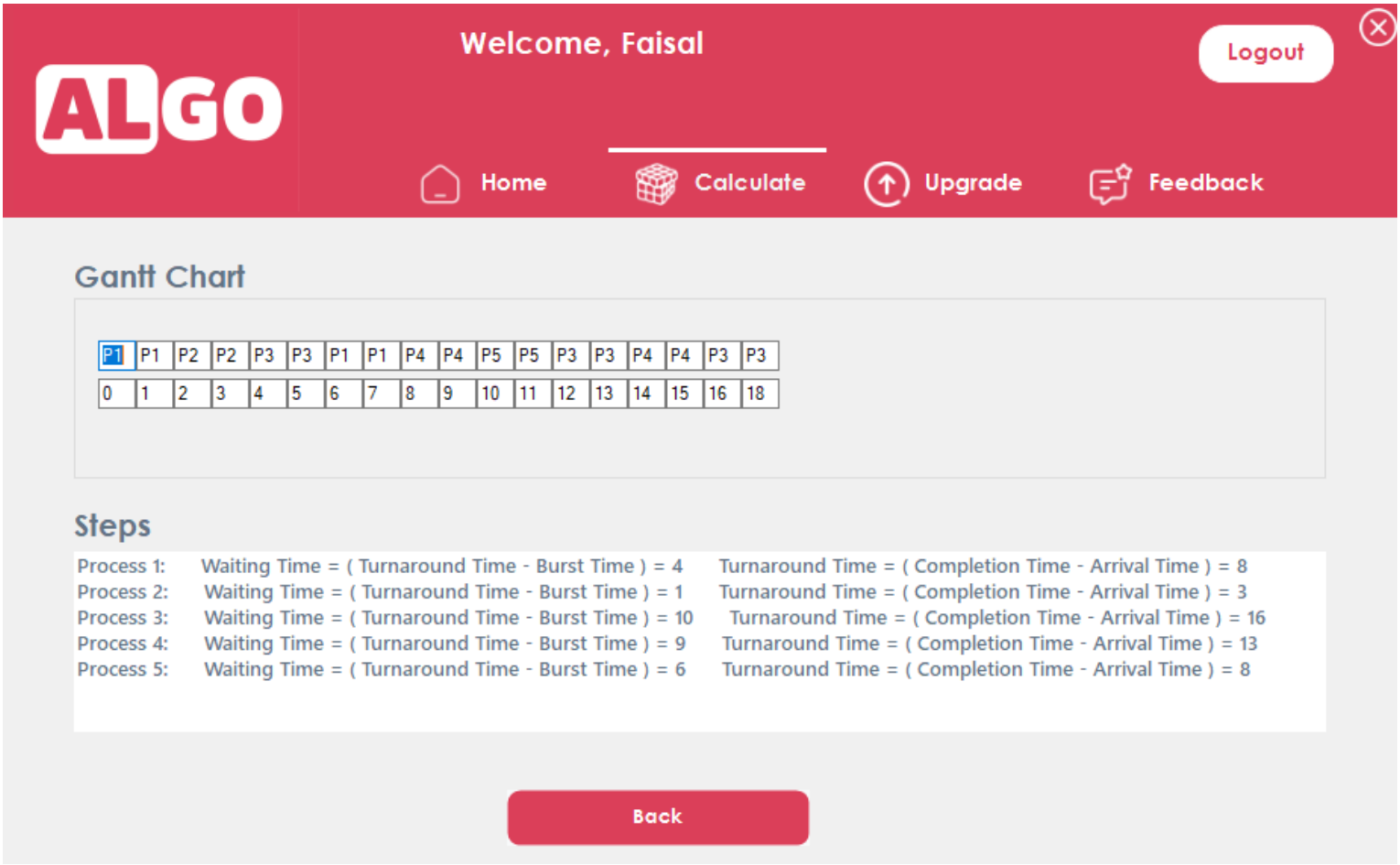
First come First Serve (FCFS) example:





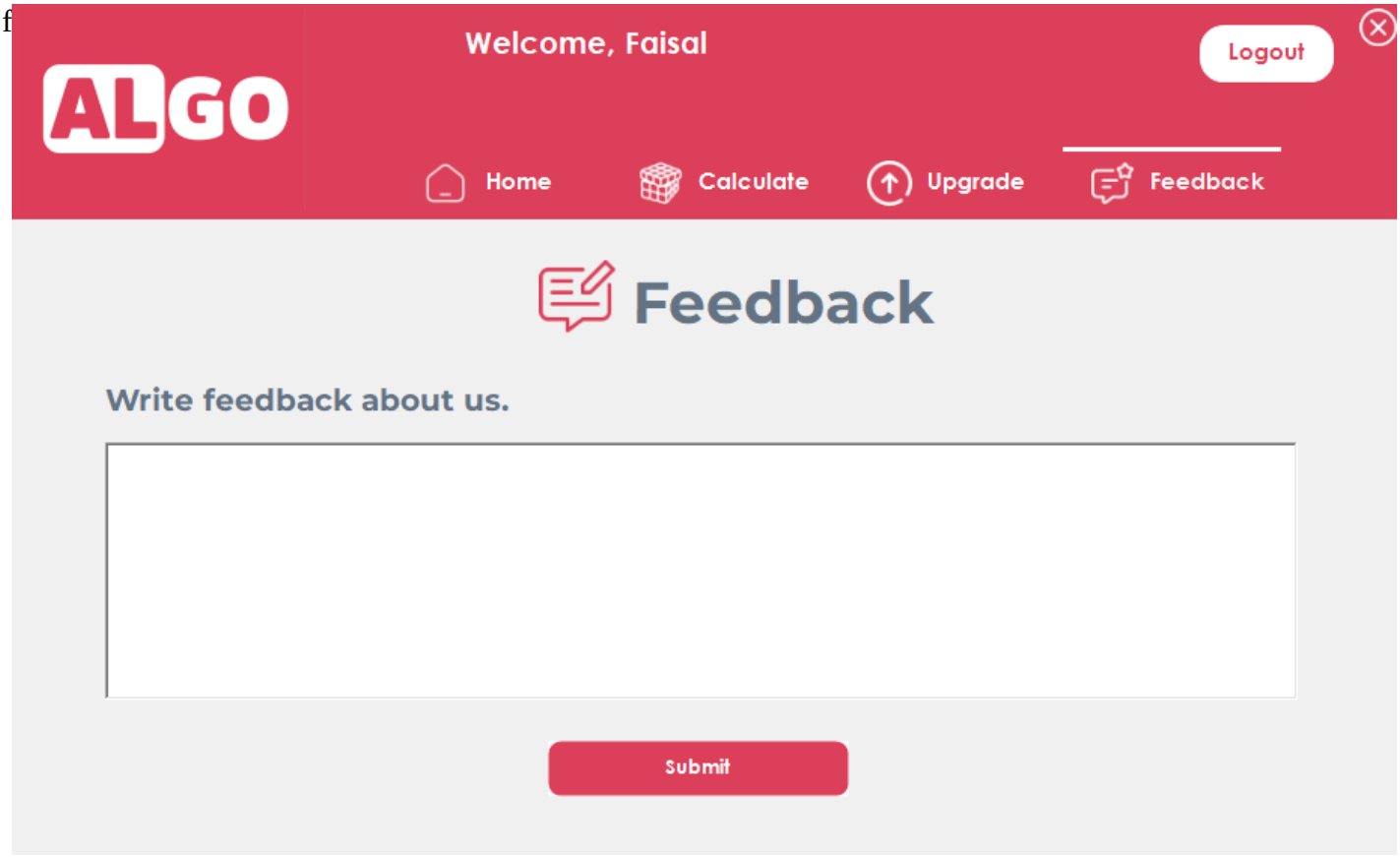
Round Robin (RR) example:





The user has the option to write feedback in the designated feedback section.

f



❖ Bash Script for Evaluating Arithmetic Expressions with a Custom Flag

1. Introduction

This project aims to develop a Bash script capable of evaluating arithmetic expressions both as separate arguments and as a single expression string. This script serves as a command-line tool for performing basic arithmetic operations such as addition, subtraction, multiplication, and division. It enhances the understanding of scripting, command-line tools, and basic arithmetic computation in the Linux environment..

2. Project Objective

The main objective of this project is to create a versatile and user-friendly Bash script that can:

- Accept arithmetic expressions as individual arguments.
- Evaluate complete arithmetic expressions provided as a single string.
- Provide clear usage instructions and examples.
- Handle different arithmetic operations accurately, including addition, subtraction, multiplication, and division.

3. Problem Statements

The project addresses the following problems:

- **Ease of Use:** Users need a simple and straightforward method to perform arithmetic calculations from the command line.
- **Flexibility:** The script should support both individual arguments for numbers and operators, as well as complete expressions provided as a single string.
- **Accuracy:** The script must handle different arithmetic operations accurately and return results with appropriate precision.
- **User Guidance:** The script should provide clear instructions and usage examples to help users understand how to use it effectively.

4. Technologies Used

The following technologies and tools were used in this project:

- **Bash Scripting:** The core of the project is written in Bash, a Unix shell and command language, enabling the creation of powerful and flexible command-line tools.
- **bc (Basic Calculator):** A command-line calculator that supports arbitrary precision arithmetic, used to evaluate expressions and perform arithmetic operations.
- **Linux Command Line:** The script is designed to run in a Linux environment, leveraging the power of the command line for user interaction and execution..

5. Command

- **Source Code**

```
faisal152@faisal152:~$ cd /usr/bin
faisal152@faisal152:/usr/bin$ sudo nano calc
faisal152@faisal152:/usr/bin$ sudo chmod 755 calc
```

```bash
#!/bin/bash

# Function to display usage
show_help() {
  echo "Usage: $0 [options] <num1> <operator> <num2>"
  echo "Options:"
  echo "  -e, --expression   Provide arithmetic expression in a single string"
  echo "  -h, --help         Show this help message and exit"
  echo
  echo "Examples:"
  echo "  $0 5 + 3"
  echo "  $0 5 '*' 3"
  echo "  $0 5 \\* 3"
  echo "  $0 -e \"(5 + 3) * 2\""
  echo "  $0 -e \"2*(6+3)\""
  echo "  $0 -e \"3+5*2-8/4\""
}
 # Function to evaluate an arithmetic expression
 evaluate_expression() {
   local expression=$1
   result=$(echo "scale=2; $expression" | bc -l)
   printf "Result: %.2f\n" "$result"
 }

 # Function to perform basic arithmetic operations
 perform_arithmetic() {
   local num1=$1
   local operator=$2
   local num2=$3

   case $operator in
     +) result=$(echo "scale=2; $num1 + $num2" | bc);;
     -) result=$(echo "scale=2; $num1 - $num2" | bc);;
     '*') result=$(echo "scale=2; $num1 * $num2" | bc);;
     /) result=$(echo "scale=2; $num1 / $num2" | bc);;
     *) echo "Invalid operator"; exit 1;;
   esac

   printf "Result: %.2f\n" "$result"
 }
# Check for flags
while [[ $# -gt 0 ]]; do
  case $1 in
    -e|--expression)
      expression_mode=true
      expression=$2
      shift 2
      ;;
    -h|--help)
      show_help
      exit 0
      ;;
    *)
      break
      ;;
  esac
done
```

```
# If expression mode is enabled, evaluate the expression
if [ "$expression_mode" = true ]; then
  evaluate_expression "$expression"
  exit 0
fi

# Check if the correct number of arguments is provided
if [ "$#" -ne 3 ]; then
  echo "Usage: $0 <num1> <operator> <num2>"
  exit 1
fi

# Perform the basic arithmetic operation
perform_arithmetic $1 "$2" $3
```

- **File location**

```
faisal152@faisal152:~$ which calc
/usr/bin/calc
```

- **Command implementation images**

```
faisal152@faisal152:/usr/bin$ calc 5 + 3
Result: 8.00
```

```
faisal152@faisal152:/usr/bin$ calc 5 - 2
Result: 3.00
```

```
faisal152@faisal152:/usr/bin$ calc 5 / 3
Result: 1.66
```

```
faisal152@faisal152:/usr/bin$ calc 5 \* 2
Result: 10.00
```

```
faisal152@faisal152:/usr/bin$ calc 5 '*' 3
Result: 15.00
```

6. Flag

- **-e or --expression Flag**

  Purpose:

  This flag allows the user to provide an entire arithmetic expression as a single string. It signals the script to evaluate the given expression directly, rather than interpreting separate arguments for numbers and operators.

  Flag implementation images:

```
faisal152@faisal152:/usr/bin$ calc --expression "4+3/2*(3+7)"
Result: 19.00
```

```
faisal152@faisal152:/usr/bin$ calc -e "(9 + 2) * 3 + 4 * 5"
Result: 53.00
```

- **--h or --help Flag**

  Purpose:

  This flag provides the user with a help message, explaining how to use the script and giving examples of valid commands. It is useful for guiding users on the correct usage of the script.

Flag implementation images:

```
faisal152@faisal152:/usr/bin$ calc -h
Usage: /usr/bin/calc [options] <num1> <operator> <num2>
Options:
  -e, --expression   Provide arithmetic expression in a single string
  -h, --help         Show this help message and exit

Examples:
  /usr/bin/calc 5 + 3
  /usr/bin/calc 5 '*' 3
  /usr/bin/calc 5 \* 3
  /usr/bin/calc -e "(5 + 3) * 2"
  /usr/bin/calc -e "2*(6+3)"
  /usr/bin/calc -e "3+5*2-8/4"
```

```
faisal152@faisal152:/usr/bin$ calc --help
Usage: /usr/bin/calc [options] <num1> <operator> <num2>
Options:
  -e, --expression   Provide arithmetic expression in a single string
  -h, --help         Show this help message and exit

Examples:
  /usr/bin/calc 5 + 3
  /usr/bin/calc 5 '*' 3
  /usr/bin/calc 5 \* 3
  /usr/bin/calc -e "(5 + 3) * 2"
  /usr/bin/calc -e "2*(6+3)"
  /usr/bin/calc -e "3+5*2-8/4"
```